Recipe Search with Ingredient Filter (API-Based) - React Project Report
Based on:

Project Overview:

A React-based recipe search application that helps users find recipes using available

Ingredients. Users can search recipes through an API, filter results by ingredients, view recipe details, and handle cases where no recipes are available.

Project Objectives:
- Build a practical React application using components.
- Implement state management using React hooks.
- Integrate an external API for dynamic recipe data.
- Apply conditional rendering and user input handling.
- Create a project suitable for GitHub hosting and portfolio use.

Features:

<u>1. Recipe Search:</u>
Users can search recipes by name or ingredient.

<u>2. Ingredient Filter:</u>
Recipes can be filtered according to available ingredients.

<u>3. Recipe Details:</u>
Display recipe information such as title, image, ingredients, and instructions.

<u>4. API Integration:</u>
Fetch recipe data from a public recipe API using useEffect.

<u>5. Conditional Rendering:</u>
Show a friendly message when no recipes match the search.

<u>6. Responsive User Interface:</u>
Clean layout suitable for desktop and mobile screens.

React Concepts Used:

- Functional Components
- Props
- useState Hook

- useEffect Hook
- Event Handling
- Lists and map() Rendering
- Conditional Rendering
- API Data Handling

Suggested Components:

- Header Component
- SearchBar Component

Ingredient Filter Component

- RecipeCard Component
- RecipeDetails Component
- Footer Component

Project Workflow:

1. User enters recipe name or ingredient.
2. React captures user input using state.
3. Application requests data from API.
4. API response is displayed as recipe cards.
5. User can view recipe details.

Development Tools:

- React JS
- JavaScript ES6
- CSS
- Node.js and npm
- Visual Studio Code
- GitHub

Career Readiness Deliverables:
- Complete React project repository
- GitHub hosted code
- Portfolio project entry
- Resume project description

Future Enhancements:
- User login system
- Save favorite recipes
- Meal planning feature
- Nutrition information

Coding:

```
import React, { useState } from 'react';

export default function RecipeSearch() {
  const [ingredient, setIngredient] = useState('');
  const [recipes, setRecipes] = useState([]);
  const [loading, setLoading] = useState(false);
  const [hasSearched, setHasSearched] = useState(false);
  const [selectedRecipe, setSelectedRecipe] = useState(null);

  // Fetch recipes by main ingredient
  const searchRecipes = async (e) => {
    e.preventDefault();
    if (!ingredient.trim()) return;

    setLoading(true);
    setHasSearched(true);
    setSelectedRecipe(null);

    try {
      const response = await fetch(
        `https://themealdb.com{encodeURIComponent(ingredient)}`
      );
      const data = await response.json();
      setRecipes(data.meals || []);
    } catch (error) {
      console.error('Error fetching recipes:', error);
      setRecipes([]);
    } finally {
      setLoading(false);
    }
  };

  // Fetch full details of a specific recipe
  const fetchRecipeDetails = async (id) => {
    try {
      const response = await fetch(
```

```
      `https://themealdb.com{id}`
    );
    const data = await response.json();
    if (data.meals) {
      setSelectedRecipe(data.meals[0]);
    }
  } catch (error) {
    console.error('Error fetching recipe details:', error);
  }
};

return (
  <div style={{ maxWidth: '800px', margin: '0 auto', padding: '20px', fontFamily: 'sans-serif' }}>
    <h1>Ingredient Recipe Finder</h1>

    {/* Search Form */}
    <form onSubmit={searchRecipes} style={{ display: 'flex', gap: '10px', marginBottom: '20px'
}}>
      <input
        type="text"
        placeholder="Enter an ingredient (e.g., chicken, tomato, rice)"
        value={ingredient}
        onChange={(e) => setIngredient(e.target.value)}
        style={{ flex: 1, padding: '10px', fontSize: '16px', borderRadius: '4px', border: '1px solid
#ccc' }}
      />
      <button
        type="submit"
        style={{ padding: '10px 20px', fontSize: '16px', backgroundColor: '#0070f3', color: 'white',
border: 'none', borderRadius: '4px', cursor: 'pointer' }}
      >
        Search
      </button>
    </form>

    {/* Loading State */}
    {loading && <p>Searching for delicious recipes...</p>}

    {/* Conditional Rendering: Main Results Layout */}
    {!loading && hasSearched && (
      <div style={{ display: 'grid', gridTemplateColumns: recipes.length > 0 && selectedRecipe ?
'1fr 1fr' : '1fr', gap: '20px' }}>

        {/* Recipe List */}
```

```
        <div>
          {recipes.length === 0 ? (
            <div style={{ padding: '20px', backgroundColor: '#fdf2f2', border: '1px solid #f8b4b4',
borderRadius: '4px', color: '#9b1c1c' }}>
              <strong>No recipes found.</strong> Try searching for another ingredient like
"chicken", "beef", or "apple".
            </div>
          ) : (
            <ul style={{ listStyle: 'none', padding: 0, margin: 0 }}>
              {recipes.map((recipe) => (
                <li
                  key={recipe.idMeal}
                  onClick={() => fetchRecipeDetails(recipe.idMeal)}
                  style={{
                    display: 'flex',
                    alignItems: 'center',
                    gap: '15px',
                    padding: '10px',
                    border: '1px solid #eee',
                    borderRadius: '8px',
                    marginBottom: '10px',
                    cursor: 'pointer',
                    backgroundColor: selectedRecipe?.idMeal === recipe.idMeal ? '#eef6ff' : 'white'
                  }}
                >
                  <img src={recipe.strMealThumb} alt={recipe.strMeal} style={{ width: '60px', height:
'60px', borderRadius: '4px', objectFit: 'cover' }} />
                  <span style={{ fontWeight: '60px' }}>{recipe.strMeal}</span>
                </li>
              ))}
            </ul>
          )}
        </div>

        {/* Recipe Details Panel */}
        {selectedRecipe && (
          <div style={{ border: '1px solid #ccc', padding: '20px', borderRadius: '8px',
backgroundColor: '#fafafa' }}>
            <h2>{selectedRecipe.strMeal}</h2>
            <img src={selectedRecipe.strMealThumb} alt={selectedRecipe.strMeal} style={{ width:
'100%', maxHeight: '200px', objectFit: 'cover', borderRadius: '4px' }} />

            <h3>Category: {selectedRecipe.strCategory} | Origin: {selectedRecipe.strArea}</h3>
```

```
            <h4>Instructions:</h4>
            <p style={{ whiteSpace: 'pre-line', lineHeight: '1.5', fontSize: '14px'
}}>{selectedRecipe.strInstructions}</p>

            {selectedRecipe.strYoutube && (
              <a href={selectedRecipe.strYoutube} target="_blank" rel="noopener noreferrer"
style={{ color: '#ff0000', fontWeight: 'bold', textDecoration: 'none' }}>

              </a>
            )}
          </div>
        )}

      </div>
    )}
  </div>
 );
}
```

Conclusion:

The Recipe Search with Ingredient Filter application combines React fundamentals, state management, API integration, and professional project practices. It satisfies the requirements of a final React development project and can be deployed online.